Tower Defense Game

Project Plan, Architecture Diagram, Game Map and Starter Code

1. Overview

A 2D, grid-based tower defense game that runs in the browser. Enemies walk a fixed path from a spawn point to your base. You spend gold to place towers on build pads, survive escalating waves, and keep your lives above zero. The first version (MVP) has one map, three tower types, three enemy types and ten waves.

2. Tech Stack

Area	Choice	Why
Language	JavaScript (ES2020)	Runs anywhere, no install for players
Rendering	HTML5 Canvas 2D	Simple, fast enough for hundreds of sprites
Build	Vite (optional)	Hot reload; plain files also work
Data	JSON for maps, towers, waves	Tune balance without touching code
Hosting	GitHub Pages or Netlify	Free static hosting

3. Project Plan

Phase	Goal	Deliverables	Est.
1. Foundation	Canvas, game loop, map	Tile grid, path drawn, fixed timestep	2 days
2. Enemies	Walk the path	Enemy class, HP bars, lives lost at base	2 days
3. Towers	Build and shoot	Pads, placement, targeting, projectiles	3 days
4. Waves + economy	Core loop playable	Wave manager, gold rewards, game over	2 days
5. Content	Variety	3 towers, 3 enemies, upgrades, 10 waves	4 days
6. UI + polish	Feels good	HUD, build menu, sounds, particles, pause	3 days
7. Test + ship	Release	Balance pass, bug fixes, deploy	2 days

4. Game Design Tables

Towers

Tower	Cost	Range	Damage	Rate	Role
Archer	50	120	10	0.6s	Cheap all-rounder
Cannon	100	100	35 (splash)	1.5s	Groups of enemies

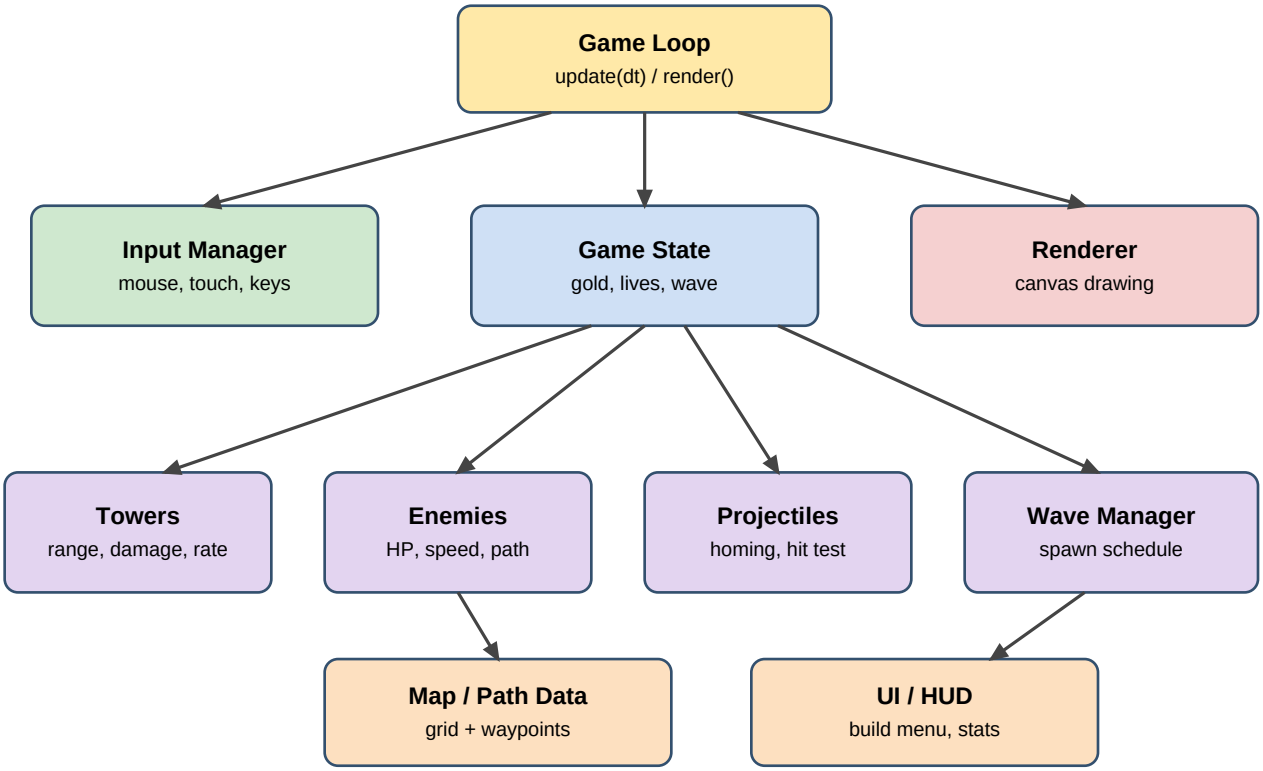
Tower	Cost	Range	Damage	Rate	Role
Frost	90	110	3 + slow 40%	0.8s	Support, slows enemies

Enemies

Enemy	HP	Speed	Reward	Notes
Grunt	30	60	5	Baseline
Runner	18	110	6	Fast, fragile
Brute	120	35	15	Slow tank, appears from wave 5

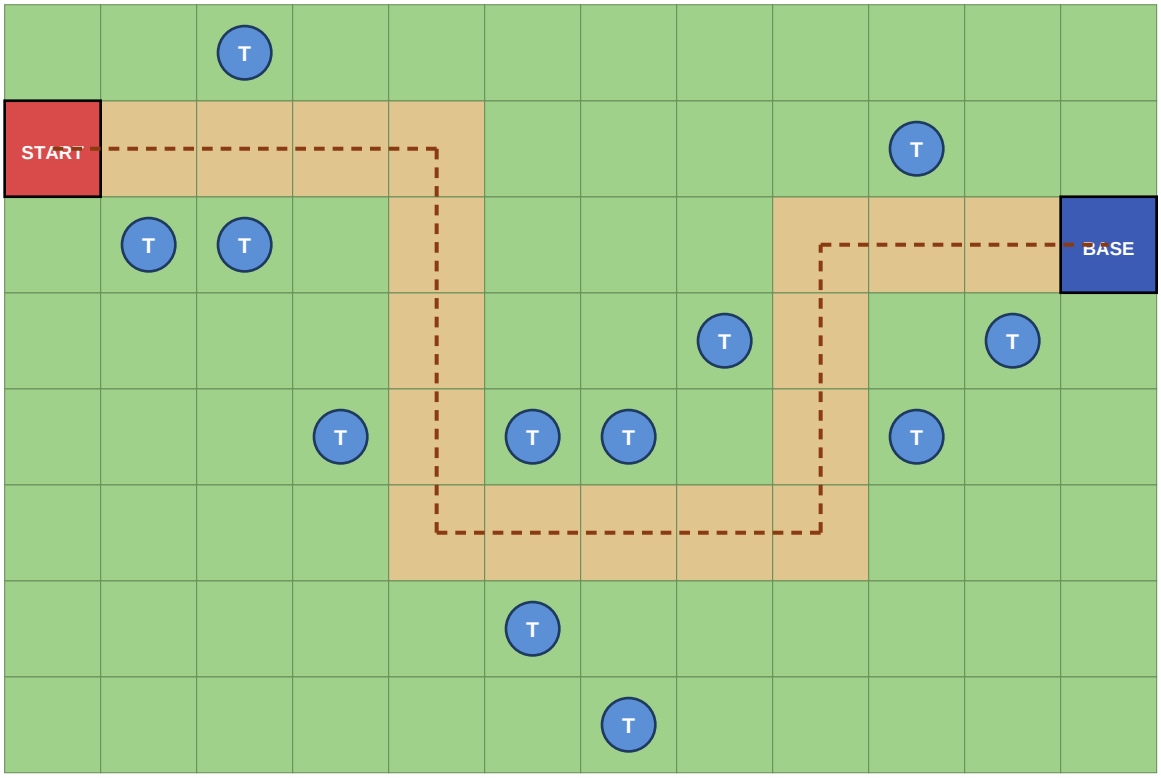
5. Architecture Diagram

The game loop drives everything. Each frame it reads input, updates the game state and entities, then renders. Map data and UI sit underneath as shared support modules.



6. Game Map: "Green Valley" (12 x 8 grid)

Tan tiles are the enemy path, blue circles marked T are tower pads, and the dashed line shows the waypoint route. Grid coordinates (column, row) start at the bottom left in this drawing; in code, row 0 is the top, so flip y when loading.



Waypoints: (0,6) to (4,6) to (4,2) to (8,2) to (8,5) to (11,5). The two long turns give towers on the inner pads overlapping coverage, and the outer pads are the cheap early picks.

7. Starter Code

Core of a playable prototype: path following, tower targeting, projectiles, waves and the main loop. Helpers such as isPad, hasTower and the draw functions are left for you to fill in (or ask me to write them next).

```
// ---------- game.js (HTML5 Canvas starter) ----------
const TILE = 48, canvas = document.getElementById('game');
const ctx = canvas.getContext('2d');
const state = { gold: 100, lives: 20, wave: 0,
                enemies: [], towers: [], shots: [] };

// Path as waypoints in grid coordinates (matches the map page)
const PATH = [[0,6],[4,6],[4,2],[8,2],[8,5],[11,5]]
  .map(([x,y]) => ({ x: x*TILE + TILE/2, y: y*TILE + TILE/2 }));

class Enemy {
  constructor(hp, speed, reward) {
    Object.assign(this, { hp, maxHp: hp, speed, reward, i: 0 });
    this.x = PATH[0].x; this.y = PATH[0].y;
  }
  update(dt) {
    const t = PATH[this.i + 1];
    if (!t) { state.lives--; this.hp = 0; this.escaped = true; return; }
    const dx = t.x - this.x, dy = t.y - this.y, d = Math.hypot(dx, dy);
    const step = this.speed * dt;
    if (d <= step) { this.x = t.x; this.y = t.y; this.i++; }
    else { this.x += dx/d*step; this.y += dy/d*step; }
  }
}

class Tower {
  constructor(x, y, range = 120, damage = 10, rate = 0.6) {
    Object.assign(this, { x, y, range, damage, rate, cd: 0 });
  }
  update(dt) {
    this.cd -= dt;
    if (this.cd > 0) return;
    // target the enemy furthest along the path within range
    let best = null;
    for (const e of state.enemies) {
      if (Math.hypot(e.x - this.x, e.y - this.y) > this.range) continue;
      if (!best || e.i > best.i) best = e;
    }
    if (best) { state.shots.push(new Shot(this, best)); this.cd = this.rate; }
  }
}

class Shot {
  constructor(from, target) {
    Object.assign(this, { x: from.x, y: from.y, target,
                          damage: from.damage, speed: 400, done: false });
  }
  update(dt) {
    const e = this.target;
    if (e.hp <= 0) { this.done = true; return; }
    const dx = e.x - this.x, dy = e.y - this.y, d = Math.hypot(dx, dy);
    if (d < 6) { e.hp -= this.damage; this.done = true;
      if (e.hp <= 0 && !e.escaped) state.gold += e.reward; return; }
    this.x += dx/d*this.speed*dt; this.y += dy/d*this.speed*dt;
  }
}

// Wave spawner: 8 + 4*wave enemies, one every 0.8s
let spawnQueue = 0, spawnTimer = 0;
function startWave() {
  state.wave++; spawnQueue = 8 + 4*state.wave;
}
```

```
function spawn(dt) {
  if (spawnQueue <= 0) return;
  spawnTimer -= dt;
  if (spawnTimer <= 0) {
    state.enemies.push(new Enemy(30 + 12*state.wave, 60 + 4*state.wave, 5));
    spawnQueue--; spawnTimer = 0.8;
  }
}

// Click a pad to place a tower (cost 50)
canvas.addEventListener('click', ev => {
  const r = canvas.getBoundingClientRect();
  const gx = Math.floor((ev.clientX - r.left) / TILE);
  const gy = Math.floor((ev.clientY - r.top) / TILE);
  if (state.gold >= 50 && isPad(gx, gy) && !hasTower(gx, gy)) {
    state.gold -= 50;
    state.towers.push(new Tower(gx*TILE + TILE/2, gy*TILE + TILE/2));
  }
});

function update(dt) {
  spawn(dt);
  [...state.enemies, ...state.towers, ...state.shots].forEach(o => o.update(dt));
  state.enemies = state.enemies.filter(e => e.hp > 0);
  state.shots   = state.shots.filter(s => !s.done);
}

function render() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  drawMap(ctx);                       // tiles, path, pads
  state.towers.forEach(t => drawTower(ctx, t));
  state.enemies.forEach(e => drawEnemy(ctx, e));
  state.shots.forEach(s => drawShot(ctx, s));
  drawHUD(ctx, state);                // gold, lives, wave
}

let last = performance.now();
function loop(now) {
  const dt = Math.min((now - last) / 1000, 0.05);  // clamp big gaps
  last = now;
  if (state.lives > 0) update(dt);
  render();
  requestAnimationFrame(loop);
}
requestAnimationFrame(loop);
```

8. Next Steps

1) Create index.html with a 576 x 384 canvas and paste in game.js. 2) Implement drawMap, isPad and hasTower using the pad list from the map page. 3) Tune wave numbers once it is playable. 4) Add the Cannon and Frost towers by extending Tower.